Practical Integration: from Zigbee to MQTT with Zigbee2MQTT

Communication Technologies for the Internet of Things, Lecture 13

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda

- The problem: connecting a Zigbee network to a message middleware
- The Zigbee2MQTT architecture
- Coordinator, installation and configuration
- Pairing and MQTT topics
- Integration, security and diagnosis

The Integration Problem (1/2)

- Lecture 07 showed that a Zigbee network has its own stack, with profiles and clusters, which IP applications do not understand.
- Lectures 11 and 12 showed that modern applications communicate by messages, through a broker.
- What is missing is the bridge: a component that translates the Zigbee world into MQTT messages, and MQTT commands into the Zigbee world.

The Integration Problem (2/2)

- Manufacturers generally offer their own proprietary bridge, which forces the use of their app and their cloud.
- An open bridge frees the user from this dependence and allows devices from different manufacturers to be mixed on the same network.

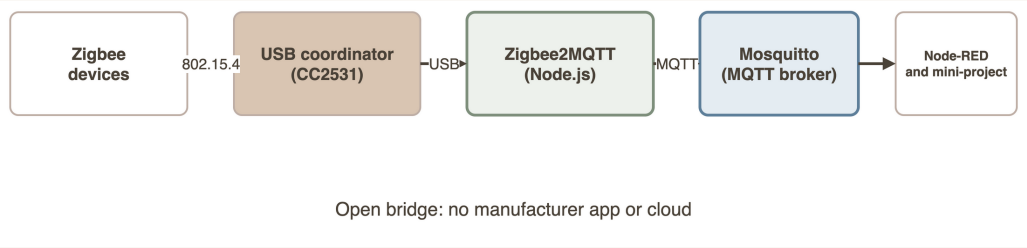
What Zigbee2MQTT Is (1/2)

- It is an open-source program, written in Node.js, that bridges a Zigbee network and an MQTT broker.
- It dispenses with manufacturers' proprietary bridges: the Zigbee device talks to a USB coordinator plugged into the computer.
- It maintains a database of supported devices, which describes what each one measures or accepts.

What Zigbee2MQTT Is (2/2)

- The state of each device is published on MQTT topics in JSON format, and commands are received on topics of their own.
- It includes a web interface for network management, pairing and viewing devices.
- Any MQTT client, such as Node-RED, can then read and control Zigbee devices.

The Integration Chain



Open bridge: no manufacturer app or cloud

The Coordinator and the Firmware (1/2)

- The coordinator is a Zigbee radio attached by USB. In the lab, the Texas Instruments CC2531 is used.
- To be used with Zigbee2MQTT, the CC2531 has to run coordinator firmware, the file `CC2531ZNP-Prod.hex`.
- Flashing the firmware requires a programmer, the CC Debugger, and a command-line tool such as `cc-tool`.

The Coordinator and the Firmware (2/2)

- The same radio can be used as a *sniffer*, with other firmware, `sniffer_fw_cc2531.hex`, to capture Zigbee traffic.
- CC2531 coordinators have capacity limits: they support few directly attached devices, and Zigbee routers should be added to extend the network.
- Recent versions of Zigbee2MQTT recommend more modern adapters; the list of supported adapters should be consulted.

Installation (1/2)

- Zigbee2MQTT runs on Linux, for example on a Raspberry Pi or a virtual machine with USB access.
- Prerequisites: an MQTT broker, such as Mosquitto, and Node.js in a supported version.
- The code is installed from the official repository, in the `/opt/zigbee2mqtt` folder, and started with the package manager indicated in the official documentation.

Installation (2/2)

- On first run, Zigbee2MQTT presents an initial setup page on port 8080, where the adapter and the broker are given.
- To run permanently, it is installed as a system service, with systemd.
- The instructions change between versions, so the official guide for the installed version is always followed.

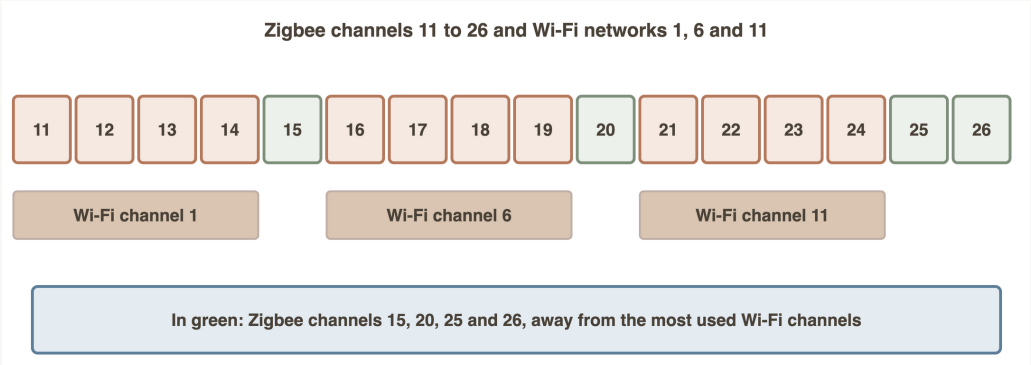
Zigbee Network Configuration (1/2)

- The `configuration.yaml` file gathers the configuration: the coordinator's serial port, the MQTT broker address and the network parameters.
- The network is defined by a PAN identifier, an extended PAN identifier, a network key and a channel.
- The channel should be chosen to avoid overlap with neighbouring Wi-Fi networks.

Zigbee Network Configuration (2/2)

- The network key protects the network traffic, and should be generated randomly and kept secret.
- The option allowing new devices to join should be on only during pairing, and turned off afterwards.
- Changing the channel or the network key requires all devices to be paired again.

Choosing the Zigbee Channel



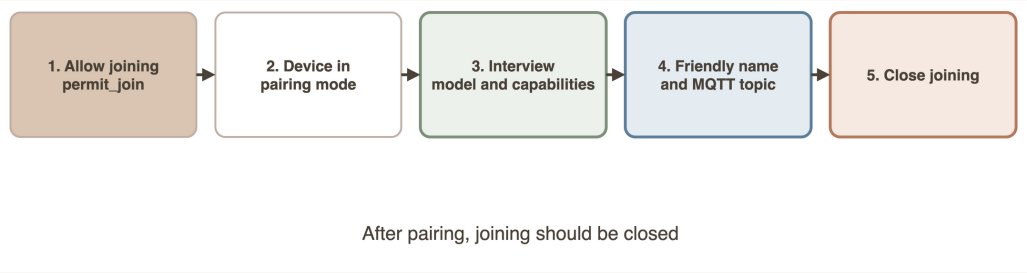
Pairing Devices (1/2)

- A device only joins the network if it is in join-permit mode.
- The device is put in pairing mode, usually by a sequence of pressing a button or power cycling.
- The coordinator then interviews the device, to discover its model and capabilities.

Pairing Devices (2/2)

- If the model is recognised, Zigbee2MQTT identifies it and starts exposing its quantities.
- Any Zigbee device can pair, even if unsupported, but only supported ones have a complete definition.
- Each device should be given a friendly name, which becomes part of the MQTT topic.

The Pairing Path



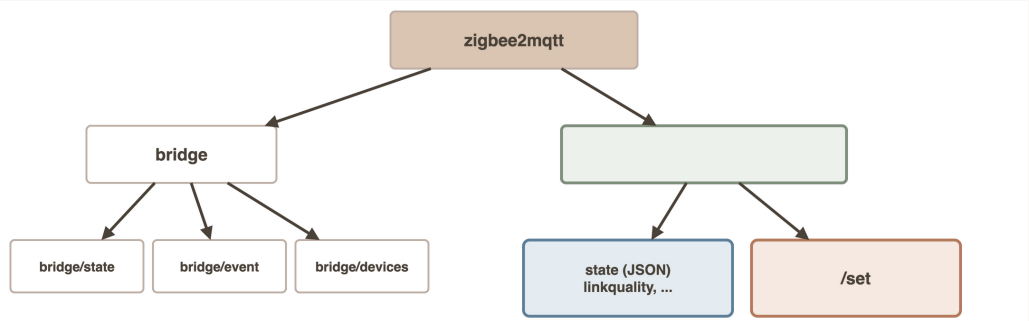
Zigbee2MQTT MQTT Topics (1/2)

- The state of each device is published on `zigbee2mqtt/<name>`, with a JSON object.
- To command a device, a JSON object is published on `zigbee2mqtt/<name>/set`.
- The bridge itself publishes its state on `zigbee2mqtt/bridge/state` and its events on `zigbee2mqtt/bridge/event`.

Zigbee2MQTT MQTT Topics (2/2)

- The link quality of each device comes in the `linkquality` field, useful for deciding where to place routers.
- State messages can be retained, through the device's `retain` option, which gives a new consumer the last known state.
- A `zigbee2mqtt/#` subscription catches everything, and is the first step in debugging the integration.

The Topic Tree



Publishing on .../set commands the device; subscribing to zigbee2mqtt/# shows everything

Integration with the Rest of the Solution

- Node-RED subscribes to the topics and turns the messages into dashboards, alarms and actions.
- In the mini-project, this bridge connects the Zigbee block to the middleware block, closing the path from sensor to dashboard.
- The same MQTT message can feed several consumers without changing the bridge, as seen in Lecture 12.

Security and Diagnosis

- Change the default network key, limit the join-permit time and protect access to the web interface.
- Protect the MQTT broker with a user and password, and do not expose it outside the local network.
- For diagnosis, consult the Zigbee2MQTT log and subscribe to `zigbee2mqtt/bridge/#`.
- For protocol-level analysis, use the sniffer and Wireshark, which needs the network key to decrypt the traffic.

Lecture Summary

- Zigbee2MQTT connects a Zigbee network to an MQTT broker, with no proprietary bridges.
- The USB coordinator, the Node.js program and the broker form the integration chain.
- Pairing is allowing entry, running the interview and naming the device.
- The zigbee2mqtt/<name> and /set topics let any MQTT client read state and send commands.

Thank you.

Questions and comments